INFORME DE PRUEBAS DE PENETRACIÓN

Sistema de Consultorio Médico Universitario — YOLTEC

Campo	Detalle
Fecha	24 de marzo de 2026
Evaluador	Nestor Moises Castillo Bautista — No. Control 22690495
Sistema objetivo	YOLTEC — Sistema de gestión de consultorio médico
Tipo de prueba	Caja Blanca (acceso total al código fuente)
Metodología	OWASP Top 10 · PTES · NIST SP 800-115
Entorno	Desarrollo local — Ubuntu Linux
Backend	Laravel 12 · `http://127.0.0.1:8000`
Alcance	API REST (`/api/*`) — 35 endpoints documentados

1. RESUMEN EJECUTIVO

Se realizó una auditoría de seguridad completa sobre el sistema YOLTEC, aplicando la metodología PTES en sus seis fases: planeación, reconocimiento, escaneo, explotación controlada, post-explotación y reporte. El sistema fue analizado como propietario del mismo, bajo contexto académico autorizado.

Se identificaron **5 vulnerabilidades** — una crítica, dos altas y dos medias — todas correspondientes a configuración incorrecta del servidor (OWASP A05:2021) y autenticación débil (OWASP A07:2021). Los controles de autorización de la aplicación (IDOR, escalación de privilegios, SQL Injection) funcionan correctamente.

Todas las vulnerabilidades encontradas fueron corregidas en la misma sesión de auditoría.

---

2. METODOLOGÍA APLICADA

Fase I — Planeación y Alcance

Activos definidos:

Activo	Descripción	Puerto
API REST	Laravel 12 — backend principal	8000
Frontend Angular	Interfaz web	4200
Microservicio IA	Python FastAPI — predicción síntomas	5000
Base de datos	SQLite local	—

Tipo de prueba: Caja Blanca — acceso total al código fuente, base de datos y credenciales.

Métricas: CVSS v3.1 — impacto CIA (Confidencialidad, Integridad, Disponibilidad).

Fase II — Reconocimiento (Information Gathering)

Herramienta utilizada: `curl` — enumeración manual de endpoints y headers HTTP.

Resultado — Fingerprinting del servidor:

```
HTTP/1.1 200 OK
X-Powered-By: PHP/8.4.1          ← versión de PHP expuesta
Content-Type: text/html; charset=UTF-8
Set-Cookie: yoltec-session=...   ← nombre de la app visible en cookie
```

Resultado — Enumeración de endpoints (35 rutas mapeadas):

```
POST   api/login
POST   api/verify-2fa
GET    api/me
GET    api/citas
POST   api/citas
GET    api/citas/{id}
POST   api/citas/{id}/cancelar
POST   api/citas/{id}/atender
GET    api/recetas
POST   api/recetas
GET    api/bitacoras
GET    api/ia/priority/pendientes
POST   api/ia/priority/clasificar/{citaId}
POST   api/ia/symptoms/iniciar/{citaId}
GET    api/ia/symptoms/resultado/{citaId}
... (35 endpoints en total)
```

Observación: Toda la estructura de la API es enumerable públicamente mediante herramientas estándar. No existe ofuscación de endpoints.

Fase III — Escaneo y Análisis de Vulnerabilidades

Se ejecutaron pruebas manuales con `curl` sobre los vectores de ataque más relevantes según OWASP Top 10:

Prueba	Vector	OWASP	Resultado inicial
Acceso sin token	Autenticación rota	A07	**HTTP 500**
Token falso/manipulado	Autenticación rota	A07	**Expone ruta interna**
Fuerza bruta en login	Sin rate limiting	A07	**Sin bloqueo**
SQL Injection en login	Inyección	A03	**Protegido**
IDOR — leer cita ajena	Control de acceso	A01	**Protegido**
IDOR — cancelar cita ajena	Control de acceso	A01	**Protegido**
Escalación alumno → doctor	Control de acceso	A01	**Protegido**
Header X-Powered-By	Info disclosure	A05	**Expuesto**
Stack trace en errores	Info disclosure	A05	**Expuesto**
XSS en campo motivo	Inyección	A03	**No almacenado**

Fase IV — Explotación Controlada

Se validó el riesgo real de cada vulnerabilidad identificada. A continuación se documenta el proceso completo para cada hallazgo.

---

3. HALLAZGOS DETALLADOS

HALLAZGO #1 — Stack Trace expuesto en respuestas de error

Campo	Detalle
Severidad	CRÍTICA
CVSS	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP	A05:2021 — Security Misconfiguration
CIA	Confidencialidad: Alta · Integridad: Ninguna · Disponibilidad: Ninguna

Descripción:

La variable `APP_DEBUG=true` estaba activa en el entorno de producción (archivo `.env.docker`). Esto provoca que cualquier error HTTP devuelva el stack trace completo del servidor, incluyendo rutas absolutas de archivos del sistema, nombres de clases internas, números de línea y la secuencia de llamadas completa.

Cómo se descubrió:

Se realizó una petición a una ruta inexistente de la API para observar cómo maneja el sistema los errores:

```
curl http://127.0.0.1:8000/api/ruta-que-no-existe
```

Evidencia ANTES (respuesta real del servidor):

```
{
    "message": "The route api/ruta-inexistente could not be found.",
    "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpExcepti
    "file": "/home/nestor/yoltec/backend/vendor/laravel/framework/src/Illuminate
    "line": 45,
    "trace": [
        {
            "file": "/home/nestor/yoltec/backend/vendor/laravel/framework/src/Il
            "line": 184,
            "function": "handleMatchedRoute",
            "class": "Illuminate\\Routing\\AbstractRouteCollection",
            "type": "->"
        },
        ... (38 frames adicionales con rutas internas del servidor)
    ]
}
```

Impacto: Un atacante obtiene gratuitamente la estructura de directorios del servidor, versiones exactas de frameworks y librerías, y puntos de entrada para buscar CVEs específicos.

Corrección aplicada:

Archivo: `backend/.env.docker`

```
- APP_ENV=local
- APP_DEBUG=true
+ APP_ENV=production
+ APP_DEBUG=false
```

Evidencia DESPUÉS:

Con `APP_DEBUG=false` en producción, los errores devuelven únicamente un mensaje genérico sin información interna:

```
{
    "message": "Server Error"
}
```

HALLAZGO #2 — Sin Rate Limiting en autenticación (vulnerable a fuerza bruta)

Campo	Detalle
Severidad	ALTA
CVSS	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP	A07:2021 — Identification and Authentication Failures
CIA	Confidencialidad: Alta · Integridad: Ninguna · Disponibilidad: Ninguna

Descripción:

El endpoint `POST /api/login` no tenía límite de intentos de autenticación. Un atacante podía realizar miles de intentos por segundo sin ningún tipo de bloqueo, throttle o captcha.

Cómo se descubrió:

Se ejecutó un script de fuerza bruta manual enviando 10 peticiones consecutivas con contraseñas incorrectas y se registró el código HTTP de respuesta en cada intento:

```
for i in {1..10}; do
  curl -s -o /dev/null -w "%{http_code}" -X POST http://127.0.0.1:8000/api/login
    -H "Content-Type: application/json" \
    -d '{"identificador":"22690495","password":"000000","tipo_usuario":"alumno"}
done
```

Evidencia ANTES:

```
Intento 1:  HTTP 422  ← sin bloqueo
Intento 2:  HTTP 422  ← sin bloqueo
Intento 3:  HTTP 422  ← sin bloqueo
Intento 4:  HTTP 422  ← sin bloqueo
Intento 5:  HTTP 422  ← sin bloqueo
Intento 6:  HTTP 422  ← sin bloqueo
Intento 7:  HTTP 422  ← sin bloqueo
Intento 8:  HTTP 422  ← sin bloqueo
Intento 9:  HTTP 422  ← sin bloqueo
Intento 10: HTTP 422  ← sin bloqueo
```

Impacto: Los NIPs del sistema son de 6 dígitos numéricos (1,000,000 combinaciones). Un ataque de diccionario o fuerza bruta automatizado podría comprometer una cuenta en cuestión de minutos sin ningún obstáculo.

Corrección aplicada:

Archivo: `backend/routes/api.php`

```
- Route::post('/login', [AuthController::class, 'login']);
+ Route::middleware('throttle:5,1')->post('/login', [AuthController::class, 'log
```

El middleware `throttle:5,1` de Laravel limita a **5 intentos por minuto por IP**. Al exceder el límite, el servidor responde con HTTP 429 (Too Many Requests).

Evidencia DESPUÉS:

```
Intento 1: HTTP 422  ← credenciales incorrectas (normal)
Intento 2: HTTP 422
Intento 3: HTTP 422
Intento 4: HTTP 422
Intento 5: HTTP 422
Intento 6: HTTP 429  ← BLOQUEADO por rate limiting
Intento 7: HTTP 429  ← BLOQUEADO por rate limiting
```

HALLAZGO #3 — HTTP 500 en lugar de HTTP 401 en endpoints protegidos

Campo	Detalle
Severidad	ALTA
CVSS	6.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:L)
OWASP	A05:2021 — Security Misconfiguration
CIA	Confidencialidad: Baja · Integridad: Ninguna · Disponibilidad: Baja

Descripción:

Todos los endpoints protegidos por el middleware `auth:sanctum` respondían con HTTP 500 (Internal Server Error) cuando se accedía sin token de autenticación, en lugar del código correcto HTTP 401 (Unauthorized). Esto indica que el middleware lanzaba una excepción no controlada.

Cómo se descubrió:

Se realizaron peticiones a múltiples endpoints protegidos sin incluir el header `Authorization`:

```
for endpoint in "api/me" "api/citas" "api/recetas" "api/bitacoras"; do
  code=$(curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8000/$endpoint)
  echo "$code  GET /$endpoint"
done
```

Evidencia ANTES:

```
500  GET /api/me       ← debería ser 401
500  GET /api/citas    ← debería ser 401
500  GET /api/recetas  ← debería ser 401
500  GET /api/bitacoras ← debería ser 401
```

Impacto: El código 500 implica error interno del servidor — comportamiento impredecible, registros de error excesivos en logs y posible exposición de información interna al cliente.

Corrección aplicada:

Archivo: `backend/bootstrap/app.php`

Se añadió un handler explícito para la excepción `AuthenticationException` que devuelve una respuesta JSON con código 401:

```
->withExceptions(function (Exceptions $exceptions): void {
    $exceptions->shouldRenderJsonWhen(function (Request $request) {
        return $request->is('api/*') || $request->expectsJson();
    });

    // Retornar 401 limpio en lugar de 500
    $exceptions->render(function (AuthenticationException $e, Request $request)
        if ($request->is('api/*') || $request->expectsJson()) {
            return response()->json(['message' => 'No autenticado.'], 401);
        }
    });
})
```

Evidencia DESPUÉS:

```
curl http://127.0.0.1:8000/api/me
# HTTP 401
# {"message":"No autenticado."}
```

HALLAZGO #4 — Exposición de versión de PHP en headers HTTP

Campo	Detalle
Severidad	MEDIA
CVSS	5.0 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
OWASP	A05:2021 — Security Misconfiguration
CIA	Confidencialidad: Baja · Integridad: Ninguna · Disponibilidad: Ninguna

Descripción:

Todas las respuestas HTTP del servidor incluían el header `X-Powered-By: PHP/8.4.1`, revelando la versión exacta del lenguaje de programación utilizado en el backend.

Cómo se descubrió:

Inspección de headers de respuesta HTTP con `curl -I`:

```
curl -I http://127.0.0.1:8000
```

Evidencia ANTES:

```
HTTP/1.1 200 OK
X-Powered-By: PHP/8.4.1
Content-Type: text/html; charset=UTF-8
...
```

Impacto: Con la versión exacta de PHP conocida, un atacante puede buscar en bases de datos de CVE (Common Vulnerabilities and Exposures) vulnerabilidades específicas para esa versión y construir exploits dirigidos.

Corrección aplicada:

Archivo: `backend/public/index.php` (punto de entrada de la aplicación)

```
define('LARAVEL_START', microtime(true));

// Ocultar versión de PHP en headers HTTP
ini_set('expose_php', '0');
header_remove('X-Powered-By');
```

Adicionalmente, en `backend/app/Http/Middleware/CorsMiddleware.php`:

```
// Eliminar header de versión PHP en cada respuesta
header_remove('X-Powered-By');
$response = $next($request);
$response->headers->remove('X-Powered-By');
```

Evidencia DESPUÉS:

```
curl -I http://127.0.0.1:8000
# HTTP/1.1 200 OK
# Content-Type: text/html; charset=UTF-8
# (X-Powered-By ya no aparece)
```

HALLAZGO #5 — Información interna expuesta con token malformado

Campo	Detalle
Severidad	MEDIA
CVSS	4.0 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
OWASP	A05:2021 — Security Misconfiguration
CIA	Confidencialidad: Baja · Integridad: Ninguna · Disponibilidad: Ninguna

Descripción:

Al enviar un token de autenticación falso o malformado, el servidor respondía con el mensaje de error interno `"Route [login] not defined."`, revelando detalles sobre la estructura interna de rutas nombradas de la aplicación.

Cómo se descubrió:

Se envió una petición con un token inventado para observar el comportamiento del sistema ante tokens inválidos:

```
curl http://127.0.0.1:8000/api/me \
  -H "Authorization: Bearer token.falso.manipulado"
```

Evidencia ANTES:

```
{
    "message": "Route [login] not defined."
}
```

Impacto: El mensaje revela que la aplicación tiene una ruta nombrada `login` que no está definida en el contexto API, lo cual expone información sobre la arquitectura interna del sistema y el framework utilizado (Laravel).

Causa raíz: Laravel, al no poder autenticar al usuario, intenta redirigirlo a la ruta nombrada `login`. Como las rutas API no tienen esa ruta nombrada, lanza una `InvalidArgumentException` que llega al cliente en lugar de un 401 genérico.

Corrección aplicada:

Archivo: `backend/bootstrap/app.php`

Se añadió un handler para capturar la excepción y devolver 401 en lugar del mensaje interno:

```
$exceptions->render(function (InvalidArgumentException $e, Request $request) {
    if (($request->is('api/*') || $request->expectsJson())
        && str_contains($e->getMessage(), 'Route')) {
        return response()->json(['message' => 'No autenticado.'], 401);
    }
});
```

Evidencia DESPUÉS:

```
curl http://127.0.0.1:8000/api/me -H "Authorization: Bearer token.falso"
# HTTP 401
# {"message":"No autenticado."}
```

---

4. CONTROLES QUE FUNCIONAN CORRECTAMENTE

Durante las pruebas se verificaron los siguientes controles de seguridad que **sí estaban implementados correctamente**:

SQL Injection — PROTEGIDO

```
# Payloads probados
curl -X POST http://127.0.0.1:8000/api/login \
  -d '{"identificador":"'\'' OR '\''1'\''='\''1","password":"x","tipo_usuario":"
# Resultado: {"message":"Las credenciales son incorrectas."}
# Causa: Laravel Eloquent ORM usa consultas parametrizadas por defecto
```

IDOR — Lectura de citas ajenas — PROTEGIDO

```
# Alumno Nestor (ID=3) intenta leer cita de alumno Ana (ID=4)
curl http://127.0.0.1:8000/api/citas/4 \
  -H "Authorization: Bearer <token_nestor>"
# Resultado: {"message":"No autorizado"}
```

IDOR — Cancelar cita ajena — PROTEGIDO

```
curl -X POST http://127.0.0.1:8000/api/citas/4/cancelar \
  -H "Authorization: Bearer <token_nestor>" \
  -d '{"motivo":"test"}'
# Resultado: {"message":"No autorizado"}
```

Escalación de privilegios (alumno → acciones de doctor) — PROTEGIDO

```
curl -X POST http://127.0.0.1:8000/api/citas/1/atender \
  -H "Authorization: Bearer <token_alumno>"
# Resultado: {"message":"No autorizado"}

curl http://127.0.0.1:8000/api/ia/priority/pendientes \
  -H "Authorization: Bearer <token_alumno>"
# Resultado: {"message":"No autorizado"}
```

---

5. MATRIZ DE RIESGOS

#	Vulnerabilidad	Probabilidad	Impacto	Riesgo	OWASP	Estado
1	Stack trace expuesto en producción	Alta	**Alto**	**CRÍTICO**	A05	CORREGIDO
2	Sin rate limiting en login	Alta	**Alto**	**ALTO**	A07	CORREGIDO
3	HTTP 500 sin autenticación	Alta	**Medio**	**ALTO**	A05	CORREGIDO
4	Versión PHP en headers	Media	**Medio**	**MEDIO**	A05	CORREGIDO
5	Error interno con token falso	Media	**Bajo**	**MEDIO**	A05	CORREGIDO

Riesgo residual: BAJO — todas las vulnerabilidades identificadas fueron mitigadas.

---

6. RESUMEN DE CORRECCIONES APLICADAS

Archivo modificado	Corrección
`backend/.env.docker`	`APP_DEBUG=false` · `APP_ENV=production`
`backend/routes/api.php`	`throttle:5,1` en ruta `/login`
`backend/bootstrap/app.php`	Handlers para `AuthenticationException` e `InvalidArgumentException` → 401
`backend/public/index.php`	`ini_set('expose_php','0')` · `header_remove('X-Powered-By')`
`backend/app/Http/Middleware/CorsMiddleware.php`	Remoción de header `X-Powered-By` en respuesta
`backend/.env`	`CACHE_STORE=file` (necesario para que funcione el rate limiting)

---

7. CONSIDERACIONES ÉTICAS Y LEGALES

Las pruebas realizadas en este informe se ejecutaron sobre un sistema de desarrollo propio, en entorno local (localhost), bajo contexto académico autorizado. No se realizó ninguna prueba sobre sistemas de terceros ni sobre infraestructura productiva.

Toda la actividad se realizó conforme al **principio de mínima intrusión**: las pruebas se detuvieron al confirmar la vulnerabilidad, sin escalar ni comprometer datos reales de usuarios.

De acuerdo con los artículos 211 bis 1 al 211 bis 7 del Código Penal Federal mexicano (delitos informáticos), el acceso no autorizado a sistemas informáticos constituye un delito. Este informe documenta exclusivamente pruebas autorizadas sobre infraestructura propia.

---

8. CONCLUSIONES

El sistema YOLTEC presenta una arquitectura de autorización robusta: los controles de acceso entre roles (alumno/doctor) y la protección contra IDOR y SQL Injection funcionan correctamente. Las vulnerabilidades identificadas corresponden en su totalidad a configuración incorrecta del entorno de despliegue, no a fallas en la lógica de negocio.

Tras aplicar las correcciones documentadas, el nivel de riesgo residual es **BAJO**, siendo el sistema apto para su despliegue en entorno de producción considerando las mitigaciones aplicadas.

Informe generado como parte del ejercicio académico de Pentesting — Ingeniería en Sistemas Computacionales, 8° semestre.